GATE CSE

20-Year Previous-Paper Topic Analysis & Preparation Blueprint

Section-wise priorities • recurring concepts • question archetypes • revision plan

Prepared from the supplied CS.zip collection of GATE CSE papers (2007–2026), with multiple sessions/sets in several years.

Important scope note

The archive contains 24 PDFs covering 2007–2026, including multiple sets for some years. Several older PDFs and some later files are scanned or have incomplete text extraction. Therefore, this document is a syllabus-aligned qualitative trend analysis and a practical topic-priority map.

Priorities reflect recurring GATE-style question archetypes, conceptual centrality, and how many other topics depend on a concept.

Executive summary

- Highest-return core: Data Structures & Algorithms, Discrete Mathematics, Operating Systems, DBMS, Computer Networks, Computer Organization & Architecture, and Theory of Computation.
- Do not neglect Engineering Mathematics, General Aptitude, or Digital Logic: they are structured scoring opportunities and can improve the overall score without requiring advanced implementation skills.
- GATE rewards precise reasoning: trace an algorithm, derive a recurrence, calculate a schedule/page fault, normalize a schema, evaluate a logic circuit, or follow a protocol—not just recall definitions.
- Study by concept clusters, then solve topic-wise PYQs, then mixed timed sets. Maintain an error log separating concept gaps, calculation errors, misread constraints, and time-management errors.

YT - Dhruv Narayan Raina

How to read the priority labels

Label	Meaning	Action
P1 — Core / very high	Recurring, foundational, and/or supports many other topics.	Learn thoroughly; solve a large PYQ set; revisit during revision.
P2 — High / important	Commonly examinable or capable of producing multi-step questions.	Master standard methods and common edge cases.
P3 — Targeted / lower relative priority	Narrower or more syllabus-specific, but still in scope.	Cover after core topics; do not leave completely untouched.

At-a-glance section priority map

Section	Priority	Highest-yield focus
General Aptitude	P1	Verbal, quantitative, analytical reasoning; accuracy and speed
Engineering Mathematics	P1	Discrete math, probability/statistics, linear algebra, calculus
Programming & Data Structures	P1	C/Python-style tracing, trees, graphs, hashing, stacks/queues
Algorithms	P1	Complexity, sorting/searching, greedy, DP, graph algorithms
Digital Logic	P1/P2	Boolean algebra,

		combinational/sequential circuits, number systems
Computer Organization & Architecture	P1	Pipelining, cache, addressing, ISA, arithmetic, memory hierarchy
Operating Systems	P1	Processes, synchronization, scheduling, deadlocks, virtual memory
DBMS	P1	SQL, relational algebra, normalization, transactions, indexing
Computer Networks	P1	Subnetting, routing, TCP/UDP, flow/congestion control, protocol math
Theory of Computation	P1	DFA/NFA/regular languages, CFG/PDA, decidability
Compiler Design	P2	FIRST/FOLLOW, parsing, syntax-directed translation, runtime
Software Engineering	P2/P3	Process models, testing, estimation, reliability, design basics
Web / emerging syllabus-specific areas	Check official syllabus	Only prepare where explicitly included in the relevant exam syllabus

YT - Dhruv Narayan Raina

1. General Aptitude

Priority: P1 — Core scoring area | This section is comparatively independent of the technical syllabus. Small improvements in speed and accuracy can be valuable.

Topic checklist

- Verbal ability: grammar, sentence completion, vocabulary in context, sentence ordering, reading comprehension, inference, tone and argument structure.
- Quantitative aptitude: percentages, ratios, averages, mixtures, profit/loss, interest, time-work, speed-distance, permutations/combinations, probability, counting, logarithms, exponents, series, basic algebra and geometry.
- Analytical and spatial reasoning: arrangements, constraints, deduction, data interpretation, charts/tables, visual patterns and elementary logical reasoning.
- Numerical accuracy: units, approximations, fractions, inequalities and sanity checks.

Typical question patterns

- Short calculation under time pressure; compare quantities or infer a missing value from a table/graph.
- Read a short passage and distinguish stated facts from implied conclusions.
- Translate a word problem into equations or constraints, then eliminate impossible options.

Preparation approach

- Practise in short timed sets 3–4 times per week rather than cramming at the end.
- Keep a formula/shortcut sheet, but learn when a shortcut is valid.
- Review every wrong answer for the root cause: vocabulary, modelling, arithmetic, or rushing.

YT — Dhruv Narayan Raina

2. Engineering Mathematics

Priority: P1 — Core | Mathematics is both a direct scoring area and the language used in algorithms, probability, digital logic, and performance analysis.

Topic checklist

- Discrete mathematics — propositional and predicate logic, equivalences, quantifiers, validity, sets, relations, functions, equivalence relations, partial orders, lattices at syllabus depth, countability basics if specified.
- Combinatorics — permutations and combinations, inclusion–exclusion, pigeonhole principle, recurrence relations, counting arguments, binomial coefficients.
- Graph theory — paths, cycles, connectivity, components, trees, bipartite graphs, degree properties, Euler/Hamilton concepts, planar graph basics and graph representations.
- Probability — conditional probability, Bayes' theorem, independence, random variables, expectation, variance, common discrete distributions and probability calculations.
- Linear algebra — matrices, determinants, rank, systems of linear equations, eigenvalues/eigenvectors and linear independence.
- Calculus — limits, continuity, differentiation, maxima/minima, integration and elementary applications within the official syllabus.
- Numerical/other mathematics — differential equations or numerical methods only where included in the applicable official syllabus.

Typical question patterns

- Determine whether a relation is reflexive, symmetric, antisymmetric, transitive, or an equivalence relation.
- Count arrangements or outcomes using a recurrence, inclusion–exclusion, or pigeonhole reasoning.
- Compute a graph property, matrix rank, eigenvalue, probability, expectation, or variance.
- Use derivatives/integrals to solve a constrained or elementary optimization/calculus problem.

Preparation approach

- Prioritize logic, relations/functions, counting, graph theory, probability, and linear algebra before low-frequency syllabus edges.
- Build a one-page formula sheet and pair every formula with one worked example and one trap case.
- For graph theory, draw small examples; for probability, define events before applying formulas; for linear algebra, verify dimensions and rank.

3. Programming, C basics & Data Structures

Priority: P1 — Core | Questions frequently test the exact state of a program or data structure after a sequence of operations. Read every boundary condition carefully.

Topic checklist

- Programming fundamentals: precedence, associativity, scope, recursion, parameter passing, pointers/references where applicable, arrays, strings, structs, bit operations, loops and nested loops.
- Language semantics: integer division, type conversion, short-circuit evaluation, side effects, static/local variables and memory layout at the level required by the paper.
- Stacks and queues: push/pop, circular queue, deque, expression evaluation, infix/postfix/prefix conversions.
- Linked lists: singly/doubly/circular lists, insertion/deletion, pointer updates, traversal and complexity.
- Trees: binary trees, BST search/insert/delete, traversals, height/depth, full/complete trees, heaps, priority queues, balanced-tree fundamentals.
- Hashing: hash functions, collisions, chaining, open addressing, linear/quadratic probing and load factor.
- Graphs: adjacency matrix/list, BFS, DFS, components, topological ordering and traversal complexity.
- Memory and data representation: arrays, pointers, recursion stack, bitwise operations and asymptotic space.

YT - Dhruv Narayan Raina

Typical question patterns

- Trace code and report output, variable values, number of iterations, or number of function calls.
- Determine stack/queue contents after operations or output of tree traversals.
- Compute tree height, number of nodes/leaves, heap state, BST property, or hash-table placement.
- Compare time and space complexity for alternative representations.

Preparation approach

- Simulate with a table: line/iteration, variable values, data structure state.
- Memorize standard traversal orders, but practise on irregular trees and graphs.
- For pointers/recursion, draw the memory/state changes; do not mentally skip steps.
- Learn complexity of each operation under average, worst-case, and stated assumptions.

4. Algorithms

Priority: P1 — Core | This is one of the most reusable sections: complexity analysis also supports data structures, graph problems, and systems performance questions.

Topic checklist

- Asymptotic analysis: O, Ω, Θ, little-o/little-omega where relevant; growth-rate ordering; best/average/worst case; amortized analysis basics.
- Recurrences: substitution, recursion tree, Master theorem and recognizing when Master theorem does not apply.
- Searching and sorting: binary search, merge sort, quicksort, heapsort, insertion/selection/bubble sort; stability, in-place property, comparison lower bounds.

- Algorithm paradigms: divide-and-conquer, greedy choice, dynamic programming, backtracking and branch-and-bound at syllabus depth.
- Greedy algorithms: activity selection, interval scheduling, Huffman coding, minimum spanning tree, shortest-path conditions.
- Dynamic programming: optimal substructure, overlapping subproblems, state definition, transitions, base cases, table order and space optimization.
- Graph algorithms: BFS/DFS, topological sort, shortest paths (BFS for unweighted graphs, Dijkstra under nonnegative weights, Bellman–Ford where negative edges matter), Floyd–Warshall, Prim, Kruskal, disjoint sets.
- Correctness: loop invariants, induction, exchange arguments and counterexamples.

Typical question patterns

- Find time complexity of nested loops with nontrivial bounds or halving/doubling variables.
- Solve a recurrence and compare growth rates.
- Trace a sorting algorithm and identify comparisons, swaps, stability, or final array state.
- Run shortest-path/MST algorithms on a small graph; identify why a greedy choice is valid or invalid.
- Construct a DP recurrence or fill a table for a small input.

Preparation approach

- Always write the input-size variable and dominant operation before simplifying complexity.
- For DP, explicitly define state, transition, base case, answer location, and time/space complexity.
- For graph algorithms, know preconditions: negative weights, disconnected graphs, directed versus undirected edges.
- Practise proving why an algorithm works, not just memorizing pseudocode.

YT - Dhruv Narayan Raina

5. Digital Logic

Priority: P1/P2 — High | Digital logic questions are often compact and mechanical once the representation and circuit state are handled correctly.

Topic checklist

- Number systems: binary, octal, decimal, hexadecimal; signed magnitude, 1’s/2’s complement; range and overflow; fixed-point representation.
- Boolean algebra: identities, De Morgan’s laws, canonical SOP/POS, minterms/maxterms, truth tables, simplification.
- Karnaugh maps: grouping, don’t-care terms, prime implicants and minimum expressions.
- Logic gates and universal gates: NAND/NOR implementations, XOR/XNOR, propagation through a combinational circuit.
- Combinational circuits: adders/subtractors, comparators, multiplexers/demultiplexers, encoders/decoders, parity circuits.
- Sequential circuits: latches, flip-flops, excitation/characteristic tables, registers, counters, timing/state transitions.
- Finite-state machines: state diagrams/tables, Mealy versus Moore, state minimization and sequence detection.
- Timing basics: propagation delay, setup/hold concepts and critical-path reasoning when specified.

Typical question patterns

- Convert numbers or determine representable range and overflow.
- Simplify a Boolean function or choose minimum gates using K-map.
- Determine output for a circuit, mux configuration, or sequence of flip-flop states.
- Find next state, output sequence, or minimum number of state bits.

Preparation approach

- Practise signed arithmetic carefully; distinguish carry-out from signed overflow.
- For sequential circuits, write present state → input → next state → output in a table.
- Use K-map adjacency correctly, including wrap-around edges and don’t-care choices.

6. Computer Organization & Architecture (COA)

Priority: P1 — Core | COA mixes conceptual understanding with numerical calculations. Cache, pipelining, memory, and instruction execution deserve particular care.

Topic checklist

- Instruction set and addressing: instruction formats, addressing modes, effective address, instruction length and execution steps.
- CPU performance: clock rate, CPI, instruction count, CPU time, speedup, Amdahl's law and performance comparisons.
- Pipelining: stages, throughput, latency, ideal speedup, structural/data/control hazards, forwarding, stalls, branch penalties and pipeline timing diagrams.
- Memory hierarchy: locality, cache mapping (direct, fully associative, set associative), tag/index/offset, hit/miss, miss penalty, AMAT, write-through/write-back, write allocate/no-write allocate.
- Main memory and virtual memory interface: address width, capacity, memory interleaving, cache blocks and hierarchy calculations.
- Computer arithmetic: signed numbers, addition/subtraction, overflow, multiplication/division algorithms and floating-point representation/precision as in syllabus.
- Control unit: hardwired versus microprogrammed control, micro-operations and control sequencing.
- I/O and buses: interrupts, DMA, polling, bus bandwidth and transfer-time calculations.

Typical question patterns

- Calculate cache tag/index/offset bits, hit ratio, average access time, or number of cache lines.
- Calculate pipeline cycles with hazards, stalls, forwarding, branches, or unequal stage assumptions.
- Compute CPU execution time or compare performance using CPI/clock/instruction count.
- Determine effective address, memory size, number of bits, or floating-point representation.

YT - Dhruv Narayan Raina

Preparation approach

- Write down assumptions first: cache policy, block size, address size, pipeline forwarding and branch model.
- Draw cache address fields and pipeline timing tables; these prevent off-by-one errors.
- Keep units consistent (bytes versus words; ns versus cycles).
- Connect formulas to definitions so that a changed assumption does not break the solution.

7. Operating Systems

Priority: P1 — Core | OS questions often combine a short scenario with exact calculations. Process-state reasoning, synchronization, scheduling, and virtual memory are core.

Topic checklist

- Processes/threads: process states, PCB, context switching, user/kernel mode, threads, concurrency and process creation concepts.
- CPU scheduling: FCFS, SJF/SRTF, priority, Round Robin, response/waiting/turnaround time, throughput and starvation.
- Synchronization: critical section, race conditions, mutex/semaphore/monitor concepts, producer–consumer, readers–writers, dining philosophers, deadlock versus starvation.
- Deadlocks: necessary conditions, resource-allocation graphs, prevention/avoidance/detection, Banker's algorithm and safe sequences.
- Memory management: contiguous allocation, fragmentation, paging, segmentation, page tables, TLB, multilevel page tables, effective access time.
- Virtual memory: demand paging, page faults, FIFO/LRU/Optimal/Clock replacement, frame allocation, Belady's anomaly and working-set/thrashing concepts.
- File systems: allocation methods, directories, inodes/metadata, free-space management and disk-block calculations.

- I/O and disks: disk scheduling (FCFS, SSTF, SCAN, C-SCAN and variants), seek distance and device handling.

Typical question patterns

- Build Gantt charts and compute waiting, turnaround and response time.
- Trace semaphores/locks and determine possible interleavings, deadlocks or race conditions.
- Run page-replacement algorithms and count page faults; track reference strings frame by frame.
- Apply Banker's algorithm or resource-allocation graph reasoning.
- Calculate effective memory access time with TLB/page-fault assumptions or disk head movement.

Preparation approach

- Use a timeline for scheduling and a frame table for paging; do not rely on mental simulation.
- State whether the question asks for waiting, turnaround, response, or completion time.
- For semaphores, track each variable and blocked process in order; distinguish atomic operations from ordinary code.
- Practise the difference between deadlock avoidance, prevention, detection, and recovery.

8. Database Management Systems (DBMS)

Priority: P1 — Core | DBMS questions reward formal reasoning: translate the schema/query correctly, then apply relational and transaction rules.

Topic checklist

- Relational model: schemas, tuples, keys, superkeys, candidate/primary/foreign keys, entity and referential integrity.
- Relational algebra: selection, projection, union, difference, Cartesian product, joins, renaming, division and expression equivalence.
- SQL: joins, nested queries, aggregation, GROUP BY/HAVING, NULL semantics, set operations, duplicates, correlated subqueries and query logic.
- Functional dependencies: attribute closure, candidate keys, minimal cover, implication and lossless/dependency-preserving decomposition.
- Normalization: 1NF, 2NF, 3NF, BCNF; identify anomalies and test decomposition properties.
- Transactions: ACID, schedules, conflict/view serializability, precedence graphs, recoverability, cascadeless/strict schedules.
- Concurrency control and recovery: locking, 2PL, deadlocks, timestamps, logging, checkpoints and crash recovery concepts.
- Indexing and storage: primary/secondary indexes, dense/sparse indexes, B/B+ trees, hashing and I/O cost estimation.

YT - Dhruv Narayan Raina

Typical question patterns

- Compute attribute closure and identify all candidate keys.
- Find the highest normal form, decompose a relation, or test lossless join and dependency preservation.
- Evaluate SQL/relational-algebra output on a small relation instance.
- Build a precedence graph and decide conflict serializability or recoverability.
- Calculate B/B+ tree height, fanout, index entries, or block I/O cost.

Preparation approach

- For keys, compute closure systematically and test minimality by removing attributes.
- For SQL, explicitly account for NULL, duplicates, grouping, and join multiplicity.
- For schedules, draw the precedence graph and distinguish conflict serializability from view serializability.
- Learn the definitions of 3NF and BCNF precisely; most errors arise from mixing their conditions.

9. Computer Networks

Priority: P1 — Core | Networking questions frequently mix protocol concepts with arithmetic: subnetting, delay, throughput, window sizes, and routing.

Topic checklist

- Layering and services: OSI/TCP-IP models, encapsulation, service versus protocol, packet/segment/frame distinctions.
- Data link layer: framing, error detection (parity, checksum, CRC), error correction basics, MAC, Ethernet, switching, ARP and VLAN concepts where included.
- IP addressing: IPv4, CIDR, subnet masks, network/broadcast addresses, host counts, fragmentation and MTU.
- Routing: forwarding versus routing, routing tables, longest-prefix match, distance-vector and link-state ideas, shortest-path routing and convergence issues.
- Transport layer: UDP/TCP, connection setup/teardown, sequence/acknowledgement numbers, sliding windows, retransmission, flow control, congestion control and throughput.
- Application layer: DNS, HTTP, email and other standard application protocols within syllabus scope.
- Performance math: propagation/transmission/processing/queuing delay, bandwidth-delay product, utilization, throughput, stop-and-wait, Go-Back-N and Selective Repeat.
- Network security basics only where explicitly listed in the relevant official syllabus.

Typical question patterns

- Find subnet/network range, broadcast address, number of subnets or usable hosts.
- Compute transmission and propagation delays, total end-to-end delay, utilization or throughput.
- Trace TCP sequence/ACK numbers, window movement, retransmissions or handshake state.
- Calculate CRC/checksum result or determine whether a received frame is valid.
- Choose a route using longest-prefix match or reason about routing algorithm behavior.

YT - Dhruv Narayan Raina

Preparation approach

- Write units and distinguish link bandwidth from end-to-end throughput.
- For subnetting, convert the prefix to a mask and count host bits explicitly.
- For sliding-window questions, draw sender/receiver windows and label sequence-number space.
- Keep propagation delay (distance/speed) separate from transmission delay (bits/rate).

10. Theory of Computation (TOC)

Priority: P1 — Core | TOC questions are highly sensitive to formal definitions. Draw automata and use language properties rather than intuition alone.

Topic checklist

- Regular languages: DFA/NFA/ε-NFA, transition tables, acceptance, equivalence, subset construction and state minimization.
- Regular expressions and closure properties: union, concatenation, Kleene star, complement, intersection and difference where applicable.
- Pumping lemma and non-regularity: choose a valid split and reason about all allowable decompositions; understand limitations of the lemma.
- Context-free grammars: derivations, parse trees, ambiguity, grammar simplification, normal forms at syllabus depth.
- Pushdown automata: stack behavior, acceptance by final state/empty stack, CFG-PDA relationship.
- Language hierarchy: regular, context-free, context-sensitive and recursively enumerable concepts at the required level.
- Decidability and computability: decidable/recognizable languages, reductions, halting problem and standard undecidability arguments.
- Closure and decision properties: know which class is closed under which operation, and where closure fails.

Typical question patterns

- Count states or determine whether a string is accepted by a DFA/NFA.
- Convert NFA to DFA, minimize a DFA, or identify an equivalent automaton.
- Determine whether a language is regular/context-free and apply an appropriate proof technique.
- Find ambiguity, derivations, parse trees, or PDA behavior for a grammar.
- Classify a problem as decidable/undecidable or follow a reduction argument.

Preparation approach

- Use formal transition tables and trace one input symbol at a time.
- Memorize closure properties in a comparison table and revisit them repeatedly.
- Do not use the pumping lemma as a universal proof method; understand what it can and cannot establish.
- For CFG questions, distinguish grammar ambiguity from multiple derivations caused by notation mistakes.

11. Compiler Design

Priority: P2 — High | Compiler questions often have a predictable mechanical workflow. FIRST/FOLLOW, parsing tables, and intermediate code are especially practice-friendly.

Topic checklist

- Compiler phases: lexical analysis, parsing, semantic analysis, intermediate representation, optimization and code generation.
- Lexical analysis: tokens, regular expressions, finite automata, longest match and lexical errors.
- Context-free grammar: left recursion, left factoring, ambiguity and grammar transformations.
- FIRST/FOLLOW sets and LL(1): nullable productions, predictive parsing table and conflict detection.
- Bottom-up parsing: shift-reduce parsing, handles, LR(0)/SLR/LR(1)/LALR distinctions and parsing-table concepts as specified.
- Syntax-directed translation: attributes, syntax trees, postfix/intermediate code, three-address code and backpatching basics.
- Runtime environment: activation records, stack frames, parameter passing, static/dynamic scope and storage allocation.
- Optimization: common subexpression elimination, constant folding/propagation, dead-code elimination, loop optimization and data-flow basics.

YT - Dhruv Narayan Raina

Typical question patterns

- Compute FIRST/FOLLOW sets and construct or inspect an LL(1) parsing table.
- Identify whether a grammar is ambiguous or needs left-recursion elimination/factoring.
- Trace shift/reduce actions or identify parsing conflicts.
- Generate three-address code or determine the effect of a local optimization.

Preparation approach

- For FIRST/FOLLOW, iterate to a fixed point and mark ε carefully.
- Separate lexical regular expressions from context-free syntax rules.
- Practise standard grammar transformations and preserve language equivalence.
- Know the purpose and limitation of each compiler phase rather than only memorizing names.

12. Software Engineering

Priority: P2/P3 — Targeted | Coverage and weight can vary with syllabus/paper structure; prepare the explicitly listed concepts after the core technical areas.

Topic checklist

- Software process models: waterfall, iterative, incremental, spiral, agile principles and risk-driven development.
- Requirements: functional/non-functional requirements, elicitation, specification, validation and traceability.
- Design: modularity, cohesion, coupling, abstraction, architecture and interface design.
- Project planning: effort/cost estimation, COCOMO-style models if included, scheduling, risk analysis and resource allocation.
- Testing: unit/integration/system/acceptance testing, black-box/white-box methods, equivalence partitioning, boundary-value analysis, coverage and regression testing.
- Reliability and quality: defect density, reliability measures, verification versus validation, maintenance and configuration management.

Typical question patterns

- Choose a process model for a scenario and justify the risk/feedback trade-off.
- Compute a simple estimation, reliability or test-coverage quantity.
- Identify test cases from equivalence classes or boundary values.
- Distinguish cohesion/coupling, verification/validation, and types of testing.

Preparation approach

- Prepare crisp definition contrasts and one example per concept.
- Practise calculation templates for estimation/reliability only if they appear in the official syllabus.
- Use the latest official syllabus to decide how much time to allocate.

Cross-section patterns worth mastering

YT — Dhruv Narayan Raina

- Counting and state tracking: stacks, recursion, automata, page replacement, CPU scheduling, TCP windows, and pipeline stalls all reward a written state table.
- Graph modelling: graph theory, BFS/DFS, MST/shortest paths, deadlock graphs, precedence graphs, and routing share representations but not necessarily the same interpretation.
- Complexity and trade-offs: algorithms, hashing, indexes, cache design, and network protocols often ask for the cost of a choice under stated assumptions.
- Boundary conditions: empty input, duplicate keys, disconnected graphs, negative edge weights, NULL values, overflow, inclusive/exclusive loop bounds, and page/reference edge cases.
- Units and assumptions: bits versus bytes, words versus addresses, cycles versus time, transmission versus propagation delay, and worst-case versus average-case complexity.
- Formal definitions: normal forms, serializability, regularity, closure properties, deadlock conditions, and signed overflow are frequently lost through vague definitions.

Suggested study order

1. Phase 1 — Foundation: Discrete Mathematics + C/programming fundamentals + Data Structures. Start an error log immediately.
2. Phase 2 — Algorithmic reasoning: Algorithms + Graph Theory + Probability/Linear Algebra. Revisit complexity analysis throughout.
3. Phase 3 — Systems: COA + Operating Systems + Computer Networks. Practise numerical/state-tracing questions.
4. Phase 4 — Formal and data-centric subjects: TOC + Compiler Design + DBMS. These benefit from precise definitions and repeatable procedures.
5. Phase 5 — Digital Logic + remaining syllabus-specific subjects + aptitude refinement. Keep aptitude practice running in parallel rather than postponing it.
6. Phase 6 — PYQ cycles: solve each available paper under timed conditions, review all options/working, and re-solve errors after 3–7 days and again after 2–3 weeks.

A practical PYQ workflow

- First pass (untimed): solve topic-wise to learn the standard archetypes. Write the concept tested beside each question.
- Second pass (timed): solve mixed sets without notes. Mark guesses separately from confident answers.
- Error log columns: year/set, subject, concept, error type, correct principle, revisit date, solved correctly on retry?
- After each paper: categorize lost marks into knowledge gap, wrong model, arithmetic, misread wording, and time pressure.
- Revision: revisit formulas and definitions with active recall. Re-solving a question from memory is more useful than merely rereading its solution.
- Final stretch: focus on high-confidence gains, mixed tests, and recurring mistakes; do not chase obscure topics at the cost of core revision.

Formula / definition checklist to keep beside your notes

Area	Must know cold
Algorithms	Common growth-rate ordering; Master theorem cases; binary search/sorting complexities; BFS/DFS; Dijkstra/Bellman–Ford/Floyd–Warshall; MST properties; DP state-transition templates.
Mathematics	Counting identities; Bayes; expectation/variance; relation properties; graph degree facts; matrix rank/determinant/eigenvalue methods.
COA	CPU time = instruction count × CPI × clock-cycle time; AMAT; cache address fields; pipeline cycle counts; speedup/Amdahl.
OS	Turnaround = completion − arrival; waiting = turnaround − CPU burst (single-burst model); page-replacement rules; TLB/EAT equations; Banker safety sequence.
DBMS	Attribute closure; key test; 3NF/BCNF conditions; lossless decomposition tests; precedence graph; ACID; index fanout/I/O estimates.
Networks	Transmission delay = bits/rate; propagation delay = distance/propagation speed; bandwidth-delay product; subnet host bits; TCP/window sequence logic.
TOC/Compiler	Automata transitions and subset construction; closure-property table; FIRST/FOLLOW rules; LL(1) conflict test; grammar transformation steps.
Digital Logic	2's-complement range; Boolean identities; K-map grouping; flip-flop characteristic/excitation tables; FSM state tracing.

VT Dhruv Narayan Raina

What this analysis does—and does not—claim

- It gives a section-wise syllabus map and high-yield question archetypes aligned to the supplied GATE CSE paper collection.
- GATE syllabi and exam patterns can change. Use the official syllabus for the target year to confirm scope, especially for software engineering or any emerging/syllabus-specific items.
- Use this as a prioritization document, not as a prediction list. Complete the full syllabus and original PYQs to avoid blind spots.

Final recommendation

For the strongest preparation loop, choose one P1 section, revise the underlying concept, solve 15–30 relevant PYQs, record every error, and retest the errors later without notes. After the first pass through all P1 subjects, move to mixed timed papers. The best topic list is only useful when paired with repeated retrieval, careful error analysis, and full-paper practice.